

Spintronics Package Description. Version 2.0

March 14, 2026

1 Introduction

Spintronics package (spintronics.py, depends on gencthy.py) is intended for direct simulation of coherent spin and charge transport in semiclassical nanostructures that combine ferromagnetic and normal metals (eventually, also half-metals: ferromagnets where only one direction of spin is present). It implements a slightly expanded version of spin quantum circuit theory as described in Nazarov and Blanter, *Quantum Transport*, Ch. 2.7. The nanostructure is modeled with a circuit that includes ferromagnetic, normal, and half-metal terminals and nodes. The circuit consists of nodes connected to the terminals and to each other. The connectors between terminals/nodes of different sorts are characterized by a finite set of conductances. In distinction from other quantum theories, the spin circuit theory is linear one. The I-V characteristics are thus linear. Most important output computed is the conductance matrix in the space of the terminals. There are also tools to assert electric and spin currents in terminals, voltages and spin accumulation in nodes, given the voltages at the terminals.

The actual use of the package starts with creating an instance of the **SpintronicCircuit** class. Then the circuit is *built* by adding the terminals, nodes, and connectors. It is so that each element of the circuit should be given a name, and addressed by this name. If you do not like giving names, make your own script to do this for you.

The computation for a circuit involves inversion of a matrix with a dimension given by the total number of matrix voltage components in all nodes. This is fast for the circuits of a reasonable size (10 elements).

2 Building a circuit

Import **SpintronicCircuit** from the package spintronics. The instance of the class, our circuit, is created by calling

SpintronicCircuit ()

This returns the class instance. There are no arguments.

The following member functions of the instance add the elements of the setup without connecting them:

addNormalTerminal(name)

Adds a normal terminal with a given name.

addFerroTerminal(name, magnetization=[0,0,1.0]):

Adds a ferromagnetic terminal of given name characterized by its magnetization direction: a unit 3D vector (if you supply a non-unit vector, it is normalized).

addHMTerminal(name, magnetization=[0,0,1.0]):

Adds a ferromagnetic terminal of given name characterized by its magnetization direction.

addNormalNode(name):

Adds a normal node of a given name.

addFerroNode(name,magnetization=[0,0,1.0]):

Adds a ferromagnetic node of a given name and magnetization.

addHMNode(name,magnetization=[0,0,1.0]):

Adds a half-metal node of a given name and magnetization.

The following function adds the connections.

addConductor(name,code,name1,name2,pars):

This adds a connector of a given name. The **code** specifies if it connects nodes or terminals. The possible values of **code** are 'TT' (connects two terminals), 'NT' (connects a node and a terminal), 'NN' (connects two nodes). The arguments **name1**, **name2** are the names of these nodes/terminals that must already be added to the circuit. The function looks at the types of the elements it attempts to connect, and important parameter **pars** should provide enough numbers characterizing the connection. Let us specify those here

1. **two normal elements:** **pars** is just a float number giving the *conductance* (must be positive)
2. **ferromagnetic and normal elements:** the conductor is characterized by 4 parameters (see Ch. 2.8 of Nazarov Blanter): G - conductance, P - polarization (defined as $P = (G_{\uparrow} - G_{\downarrow})/G$, $G_{\uparrow,\downarrow}$ being partial conductances of majority/minority spins), $\text{Re}G_{\uparrow\downarrow}$ - real part of the mixing conductance, $\text{Im}G_{\uparrow\downarrow}$ - imaginary part of the mixing conductance. To remind, real part of mixing conductance describes relaxation of spin components perpendicular to magnetization direction of the ferromagnet, while the imaginary part describes precession around this magnetization. $G, \text{Re}G_{\uparrow\downarrow}$ must be positive, $|P| \leq 1$.
3. **half-metal and normal elements:** very similar to the previous item, except there are only 3 parameters, since $G_{\downarrow} = 0$, $P = 1$ for a half-metal.
4. **two ferromagnetic elements** We assume a tunnelling model of an interface between two ferromagnets, which is characterized by 6 parameters. Four parameters describe conductances G_p, G_{ap} and spin polarizations P_p, P_{ap} in parallel and anti-parallel configurations, respectively. Two remaining parameters, G_{r1}, G_{r2} describe the spin-flip reflection from the

interface in the first and second ferromagnet, respectively. To give concrete formulas, the currents for majority/minority spins in the first/second ferromagnet are given by (θ being the angle between the magnetizations).

$$\begin{aligned}
I_{\uparrow}^{(1)} &= G_B \cos^2 \frac{\theta}{2} (V_{\uparrow}^{(2)} - V_{\uparrow}^{(1)}) + G_{BS} \sin^2 \frac{\theta}{2} (V_{\downarrow}^{(2)} - V_{\uparrow}^{(1)}) + \frac{G_{r1} \sin^2 \theta}{2} (V_{\downarrow}^{(1)} - V_{\uparrow}^{(1)}) \\
I_{\downarrow}^{(1)} &= G_S \cos^2 \frac{\theta}{2} (V_{\downarrow}^{(2)} - V_{\downarrow}^{(1)}) + G_{SB} \sin^2 \frac{\theta}{2} (V_{\uparrow}^{(2)} - V_{\downarrow}^{(1)}) + \frac{G_{r1} \sin^2 \theta}{2} (V_{\uparrow}^{(1)} - V_{\downarrow}^{(1)}) \\
I_{\uparrow}^{(2)} &= G_B \cos^2 \frac{\theta}{2} (V_{\uparrow}^{(1)} - V_{\uparrow}^{(2)}) + G_{SB} \sin^2 \frac{\theta}{2} (V_{\downarrow}^{(1)} - V_{\uparrow}^{(2)}) + \frac{G_{r2} \sin^2 \theta}{2} (V_{\downarrow}^{(2)} - V_{\uparrow}^{(2)}) \\
I_{\downarrow}^{(2)} &= G_S \cos^2 \frac{\theta}{2} (V_{\downarrow}^{(1)} - V_{\downarrow}^{(2)}) + G_{BS} \sin^2 \frac{\theta}{2} (V_{\uparrow}^{(1)} - V_{\downarrow}^{(2)}) + \frac{G_{r2} \sin^2 \theta}{2} (V_{\uparrow}^{(2)} - V_{\downarrow}^{(2)})
\end{aligned}$$

The conductances G_p, G_{ap} and the polarizations P_p, P_{ap} are defined as

$$G_p = G_B + G_S; \quad G_{ap} = G_{SB} + G_{BS}; \quad P_p = \frac{G_B + G_S}{G_p}; \quad P_{ap} = \frac{G_{SB} - G_{BS}}{G_{ap}}$$

The order of parameters in the argument is as follows:

$pars = [G_p, P_p, G_{ap}, P_{ap}, G_{r1}, G_{r2}]$. For a symmetric conductor, $G_{r1} = G_{r2}$, $P_{ap} = 0$.

5. **two half-metal elements** just a single parameter, conductance in parallel configuration.
6. **half-metal and ferromagnetic elements** three parameters: G_p, G_{ap} - conductances in parallel/antiparallel configurations, G_r - spin-flip reflection at the ferromagnetic side.

In addition, one can add elements that belong to a single node although are regarded as connectors.

addRelaxation (name, nodename, conductance)

This gives a possibility to account for spin relaxation either in a normal or in a ferromagnetic node **nodename**. The conductance of such "conductor" for the relaxation rate $1/\tau_{sf}$ and the average energy splitting δ_S in the node (that is inversely proportional to the node volume) is given by (see Eq. in p. 191, Ch. 2.7)

$$G = G_Q \frac{\hbar}{\tau_{sf} \delta_S}. \quad (1)$$

addPrecession (name, nodename, pars)

This may account for the precession of spin in the normal node induced by an external magnetic field in the direction $\vec{n}$ ($\vec{n}$ being a 3D unit vector). The conductance reads (see Eq. in p. 191, Ch. 2.7)

$$G = G_Q \frac{\mu_B |\vec{B}|}{\delta_S}. \quad (2)$$

pars comprises 4 parameters: this conductance and 3 components of the unit vector $\vec{n}$.

Importantly, the circuit should be finalized after adding all elements.

finalize()

The build-up cannot be changed after that, but the parameters of the existing elements can be changed or set anew.

3 Setting and changing the parameters

This is accomplished by the following functions:

setMagnetization(name, code, magnetization):

Sets the magnetization of the ferro/half-metal node/terminal. The argument **code** specifies whether this is a node or terminal: 'N' and 'T' are two possible values.

setConductance(name, pars)

Sets the conductance parameters of the named connector. The number of floats in **pars** should correspond to the type of the connector (see **addConductor()**).

4 Getting the results

After changing the parameters, one should call **compute()**: this computes two important members of the class, the matrices **ConductanceMatrix** and **GeneralConductanceMatrix**. They in principle contain all results for given settings of conductances and magnetizations. (**finalize()** contains a call to **compute()**)

ConductanceMatrix:

If N_t is the number of terminals, this is a $N_t \times N_t$ matrix that relates the N_t -dimensional vector of currents in the terminals to N_t -dimensional vector of voltages. The order of terminals is the order of their addition. To recall the order, one may want to print the keys of the dictionary **dics.terminals**.

GeneralConductanceMatrix:

To understand the structure of this matrix, let us recall that a terminal of each sort supports n components of matrix current/voltage. ($n = 4$ for normal terminals, $n = 2$ for ferromagnetic ones, $n = 1$ for half-metal ones). The dimension of this matrix is the total number of these components, and it relates all matrix currents to all matrix voltages. The order of indices correspond to the order of terminals.

For convenience, there are helper functions for elements or submatrices of these matrices.

CMel(name1,name2):

returns the element of conductance matrix between the named terminals

GCMsub(name1,name2): returns the submatrix of the general conductance matrix corresponding to the named terminals. The dimension of the matrix

corresponds to the dimensions of the terminals. For instance, if **name1** is a normal terminal, and **name2** is a ferromagnetic terminal, a 4×2 matrix is returned.

resistanceMatrix(ground=name): This function returns the resistance matrix obtained by inversion of the conductance matrix assuming that the terminal **name** is grounded. This matrix is convenient if the setup is current-biased rather than voltage-biased.

There is also a set of convenience tools to inspect currents, general currents, and general voltages in the nodes at a given setting of voltages in all terminals. The default setting is the unit voltage at the first defined terminal, and zero voltage at the rest. This setting can be changed by

setVoltage(name,value): This sets the named terminal to the voltage given by **value**. The function also works with the lists of names and values, to simplify the setting of big number of terminals.

To come down to currents and generalized currents, one calls

computeCurrents(): This computes and stores the result in **Currents**, **GenCurrents**. The following helper functions return current/generalized current in the named terminal

current(name):

gencurrent(name):

Similarly, to assert the voltages in the Nodes, one calls

computeNodeVoltages():

The helper function

nodevoltage(name): returns the general voltage of the named node (4 values in the case of a normal node).

Check the example file *SpinExamples.py* for examples of building and use. Happy computing!